# CloudScale: Campus Lost & Found

Enterprise Cloud Architecture, Automated Scaling & AI Integration on AWS
**Course:** CS-4075 Cloud Computing | **Framework:** AI-DLC (AI-Driven Development Life Cycle)

---

**Authors / Students:** Adnan Ahmed & Huzaifa Tahir

**Region:** us-east-1 (N. Virginia)

**Application URL:**
https://d2oszhh0246ohl.cloudfront.net

**IaC Framework:** Terraform (v1.6+)

**Repository:**
https://github.com/Adnan75500/cloudscale-cs4075

**Date:** November 10, 2026

## 1. Executive Summary & Application Description

**CloudScale** is a scalable, resilient, multi-tier web application designed for university campus communities to report and retrieve lost items. The platform enables authenticated students and staff to post lost or found items with image attachments, automatically generates optimized mobile-friendly thumbnails, coordinates decoupled email notifications when someone responds to a listing, automatically categorizes items using machine learning, and drafts concise, high-quality listing descriptions using generative foundation models.

**Why this application was chosen:** A campus lost-and-found board represents an ideal real-world workload for modern cloud architectures. It features unpredictable traffic spikes (e.g., examination weeks, campus festivals, end of semester), asynchronous event-driven media processing (image uploads requiring thumbnails), transactional communication pipelines (decoupled response alerts), and specific machine learning workloads well suited for custom serverless inference (fixed category classification) combined with frontier generative models (contextual text generation).

## 2. Architecture Diagram & End-to-End Request Flow

The architecture is provisioned entirely via declarative Terraform infrastructure-as-code, following least-privilege security, high availability across two Availability Zones, and zero SSH access (leveraging AWS Systems Manager Session Manager).

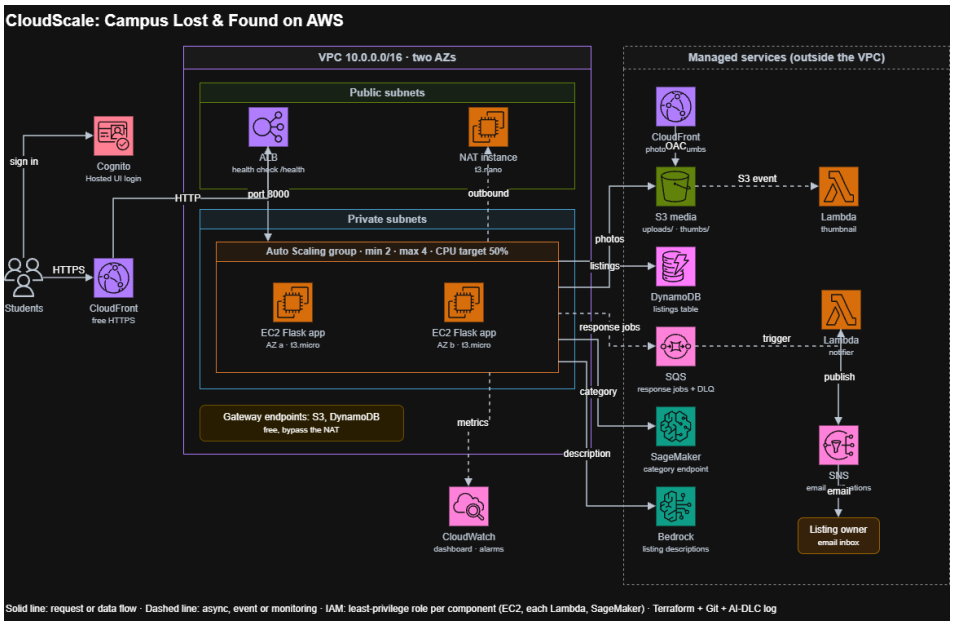



***Figure:*** *Figure 1: Full AWS System Architecture (Draw.io / PNG export)*

**Walkthrough of a Request:**

1. **Client Ingress & Edge Delivery:** A user navigates to the HTTPS CloudFront URL (`https://d2oszhh0246ohl.cloudfront.net`). CloudFront terminates TLS using its default edge certificate and forwards traffic over HTTP to an internal Application Load Balancer (ALB) across two public subnets.
2. **Compute & Session Management:** The ALB evaluates health checks (`/health`) and routes traffic across an Auto Scaling Group (ASG) of 2 to 4 private `t3.micro` EC2 instances running Gunicorn/Flask. Flask signs session cookies using an encrypted secret securely retrieved from AWS Systems Manager Parameter Store.
3. **Authentication:** When an unauthenticated user logs in, Flask redirects to Amazon Cognito Hosted UI using an OAuth 2.0 Authorization Code grant. Upon authentication, the user returns to `/callback`, and the backend validates the ID token against Cognito's JSON Web Key Set (JWKS).
4. **Storage & Serverless Media Processing:** Creating a listing uploads photos directly to the private S3 media bucket under `uploads/`. An S3 `ObjectCreated` event triggers **Lambda #1 (Thumbnail)**, which resizes the image to a 300px JPEG via a Pillow layer and writes it to `thumbs/`. Clients retrieve images exclusively through a dedicated CloudFront media distribution secured by Origin Access Control (OAC).
5. **Data Persistence:** Listing metadata (title, category, timestamps, responder threads) is written to an on-demand Amazon DynamoDB table (`cloudscale-listings`) traversing a private VPC Gateway Endpoint without touching the internet.
6. **Asynchronous Decoupled Notifications:** When a user responds to an item, Flask appends the thread to DynamoDB and enqueues a lightweight message into Amazon SQS. **Lambda #2 (Notifier)** consumes the message in batches of 1 and publishes to an Amazon SNS topic, sending a filtered email alert directly to the listing owner.
7. **AI & Machine Learning Assistance:** In the listing form, the user can click *'Suggest category'* to query a serverless **SageMaker inference endpoint** running a custom Scikit-learn TF-IDF classifier. Clicking *'Write description'* invokes **Amazon Bedrock** (`amazon.nova-lite-v1:0`) via Bedrock Converse to produce a friendly 2-3 sentence description.

# 3. Detailed Implementation by Phase

## Phase 1: Networking & Security Perimeter

**What was built:** Custom VPC (10.0.0.0/16) across 2 Availability Zones (us-east-1a, us-east-1b) featuring 2 public subnets, 2 private subnets, Internet Gateway, a cost-effective t3.micro NAT instance, and free VPC Gateway Endpoints for S3 and DynamoDB.

**Architectural Justification:** Provides network isolation so database queries and application servers never touch public IPs. Gateway endpoints keep high-volume storage traffic entirely off the NAT instance.

**Key Configuration:** `CIDR: 10.0.0.0/16; Public subnets: 10.0.0.0/24, 10.0.1.0/24; Private subnets: 10.0.10.0/24, 10.0.11.0/24. ALB SG allows only CloudFront origin prefix list. App SG accepts port 8000 only from ALB SG. NAT instance uses iptables masquerading.`

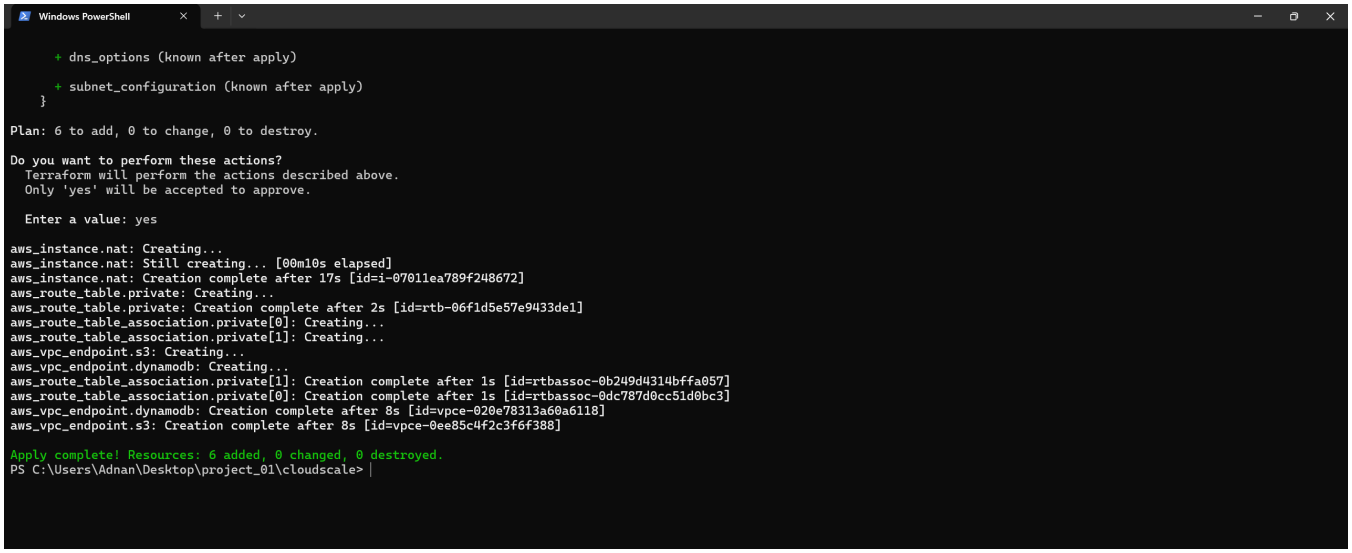



***Figure:*** *Evidence for Phase 1: Networking & Security Perimeter*

## Phase 2: Compute, Load Balancing & Auto Scaling

**What was built:** Application Load Balancer (ALB) in public subnets with target group (port 8000, /health check), CloudFront distribution providing free HTTPS without a custom domain, and an EC2 Auto Scaling Group (min 2, desired 2, max 4, t3.micro) in private subnets with CPU target-tracking at 50%.

**Architectural Justification:** Ensures zero downtime, graceful failover across AZs, horizontal scale-out under heavy campus load, and HTTPS termination required by OAuth/Cognito.

**Key Configuration:** `CloudFront origin protocol policy: http-only; ALB listener port 80; Launch template AL2023 with user-data installing Python 3.11, Gunicorn, systemd unit; TargetTrackingScaling on ASGAverageCPUUtilization = 50.`

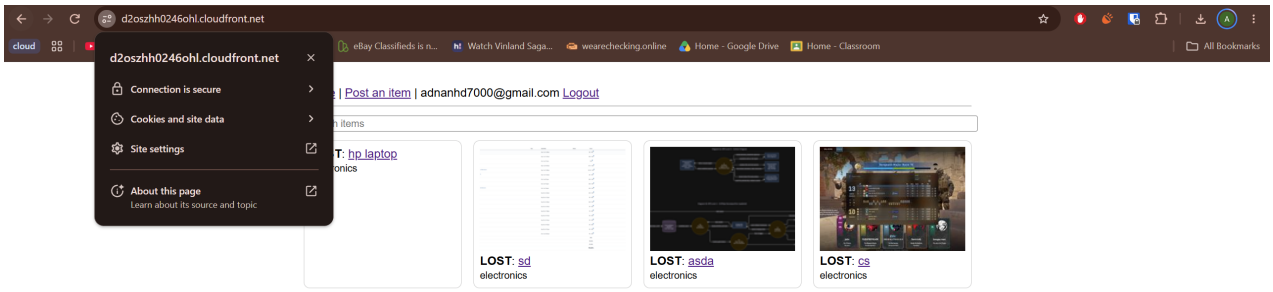



***Figure:*** *Evidence for Phase 2: Compute, Load Balancing & Auto Scaling*

## Phase 3: Storage (S3) & CDN (CloudFront Media)

**What was built:** Private S3 bucket (cloudscale-media-) with versioning enabled, lifecycle transition to Standard-IA after 30 days, noncurrent version expiration after 90 days, and a dedicated CloudFront distribution with Origin Access Control (OAC).

**Architectural Justification:** Protects against accidental deletions/overwrites, optimizes long-term storage costs for dormant photos, accelerates photo downloads globally, and guarantees no direct public S3 access.

**Key Configuration:** `S3 Bucket policy enforces AWS:SourceArn = CloudFront distribution ARN; Public Access Block enabled (100% blocked); S3 PutObject restricted to uploads/* only for EC2 role.`

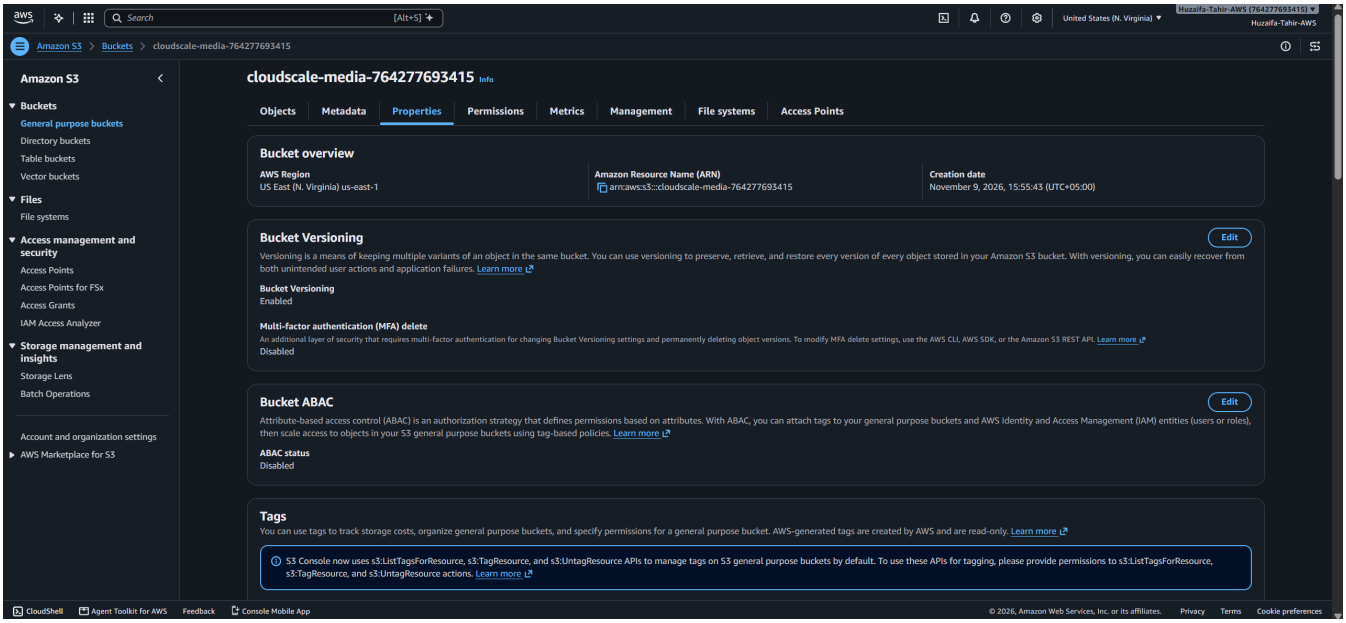



***Figure:*** *Evidence for Phase 3: Storage (S3) & CDN (CloudFront Media)*

## Phase 4: Data Layer (DynamoDB NoSQL)

**What was built:** On-demand Amazon DynamoDB table (cloudscale-listings) with partition key listing_id (String). Private VPC Gateway Endpoint attached to private route tables.

**Architectural Justification:** Lost-and-found items have flexible attributes (optional photos, variable notes, growing response threads) and require fast single-key lookups. On-demand billing costs zero dollars while idle.

**Key Configuration:** `Hash Key: listing_id (S); BillingMode: PAY_PER_REQUEST; IAM policy restricts EC2 role to PutItem, GetItem, UpdateItem, Scan, DeleteItem on table ARN only.`

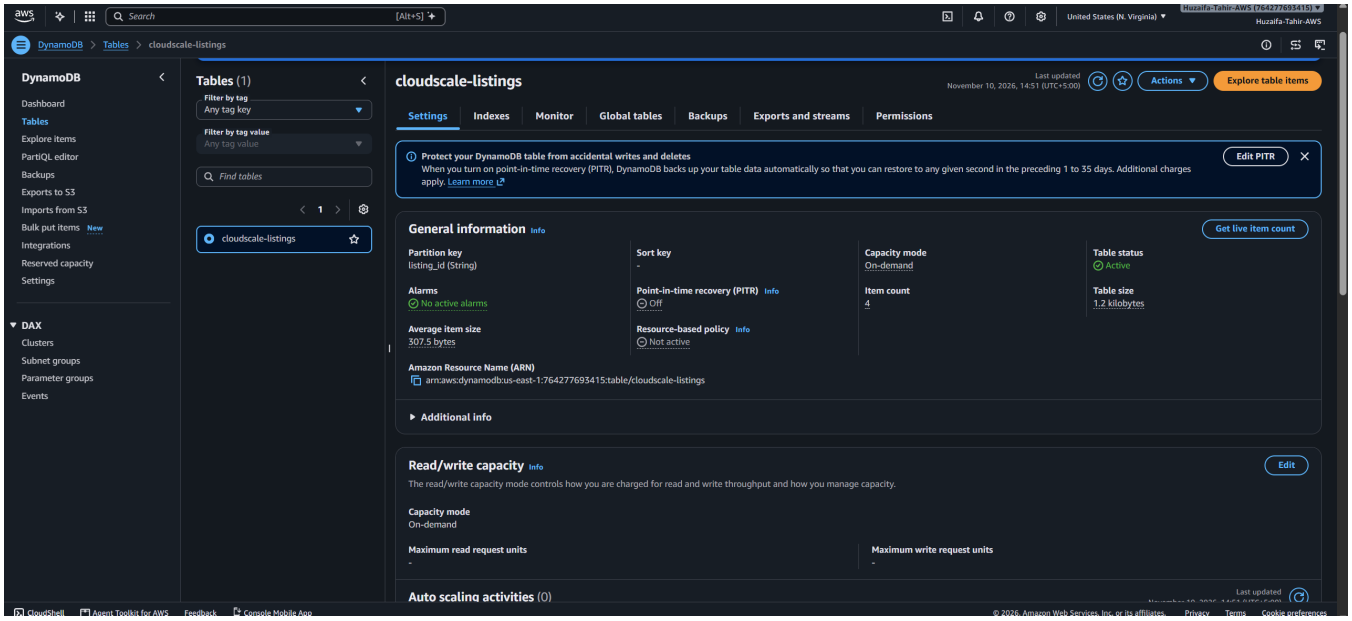



***Figure:*** *Evidence for Phase 4: Data Layer (DynamoDB NoSQL)*

## Phase 5: Identity & Access (Amazon Cognito)

**What was built:** Cognito User Pool (cloudscale-users) with self-service email sign-up and verification, hosted-UI domain, and a public app client using the OAuth 2.0 authorization code grant.

**Architectural Justification:** Provides secure identity management without managing password hashes, salt storage, or credential resets on EC2. A public client avoids leaking client secrets in user-data.

**Key Configuration:** `Username: email; Flow: code; Allowed scopes: openid, email; Callbacks: https://d2oszhh0246ohl.cloudfront.net/callback; Logout: https://d2oszhh0246ohl.cloudfront.net/.`

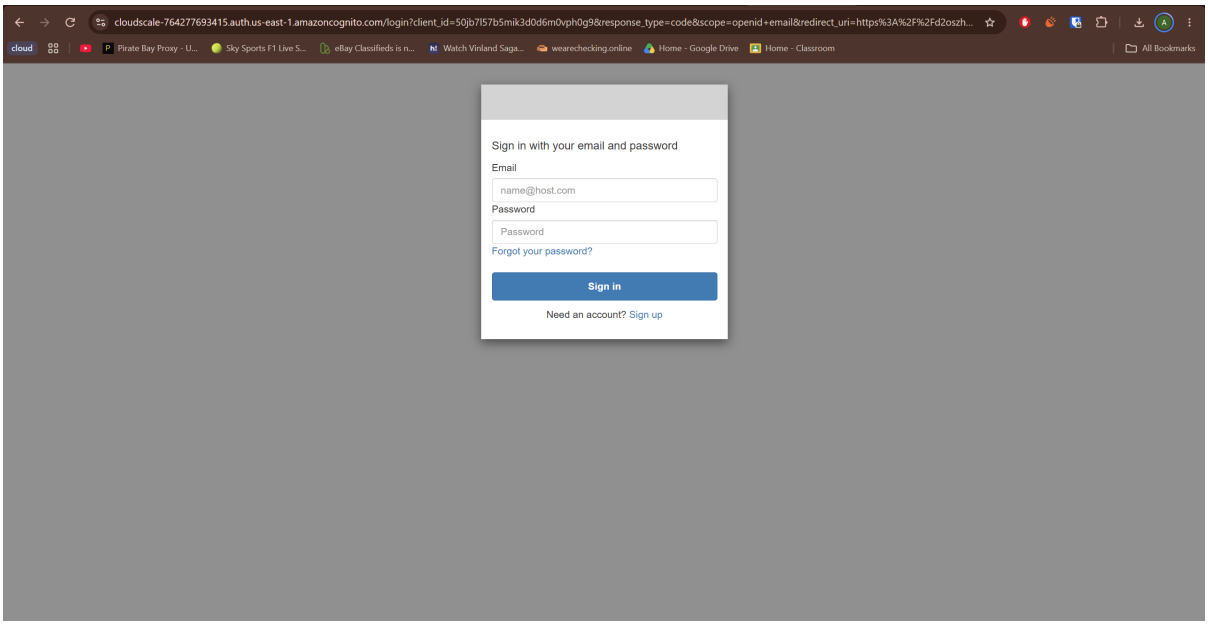



***Figure:*** *Evidence for Phase 5: Identity & Access (Amazon Cognito)*

## Phase 6: Web Application Architecture (Flask)

**What was built:** Production-ready Python Flask application served via Gunicorn (2 worker processes), integrated with SSM Parameter Store for secure cookie encryption, DynamoDB for listings, and S3 for photo uploads.

**Architectural Justification:** Delivers the end-user lost-and-found board, search filtering, session persistence across multiple EC2 instances, and responsive HTML interface.

**Key Configuration:** `Environment passed via /etc/app.env rendered dynamically from Terraform app_env locals; Session cookies: Secure, HttpOnly, SameSite=Lax; SSM parameter /cloudscale/flask-secret-key.`

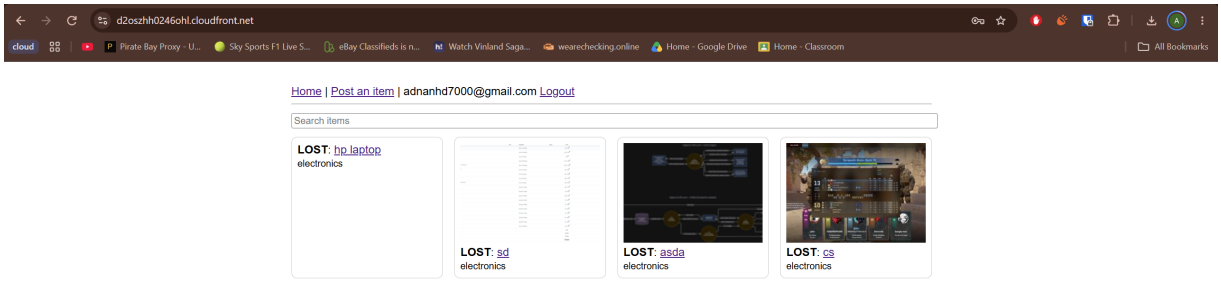



***Figure:*** *Evidence for Phase 6: Web Application Architecture (Flask)*

## Phase 7: Asynchronous Serverless (Lambda, SQS, SNS)

**What was built:** Two event-driven Lambdas: Lambda #1 (Thumbnail) triggered by S3 uploads/ with Pillow layer writing to thumbs/; SQS FIFO/Standard queue with Dead-Letter Queue (DLQ); Lambda #2 (Notifier) consuming SQS and publishing to SNS email with filter policy.

**Architectural Justification:** Decouples slow external operations (image transcoding, email delivery) from the synchronous web request loop. If SNS fails, SQS retries up to 3 times before moving messages to DLQ.

**Key Configuration:** `Thumbnail Lambda: Python 3.12, 512 MB, timeout 30s; SQS queue: visibility 60s, maxReceiveCount 3; Notifier Lambda: batch size 1; SNS topic: cloudscale-response-notifications.`

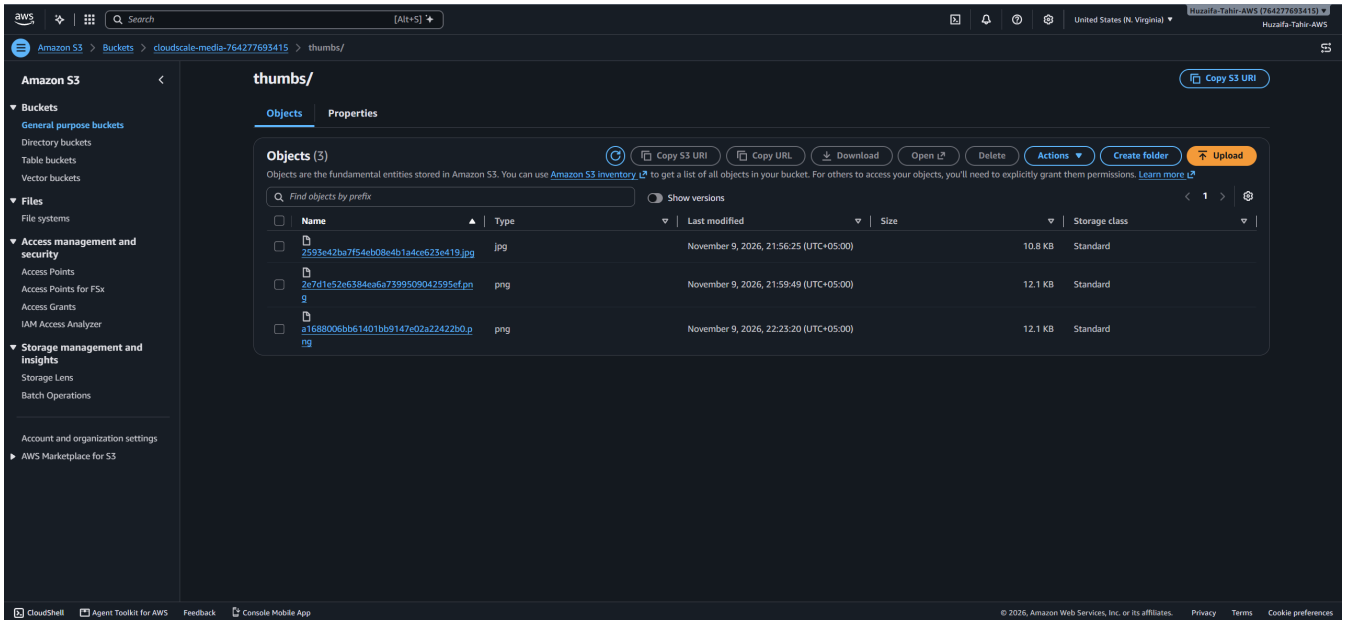



***Figure:*** *Evidence for Phase 7: Asynchronous Serverless (Lambda, SQS, SNS)*

## Phase 8: Monitoring, Telemetry & Budget Controls

**What was built:** Unified CloudWatch Dashboard (cloudscale) tracking EC2 CPU utilization, ALB RequestCount, TargetResponseTime, HealthyHostCount, and Lambda invocations/errors. CloudWatch metric alarms for high CPU and ALB 5XX errors routed to SNS alert email.

**Architectural Justification:** Provides full-stack observability, SLA validation, automated operational alerting, and budget overspend protection.

**Key Configuration:** `High CPU Alarm: CPUUtilization > 70% for 3 evaluation periods (60s); ALB 5XX Alarm: HTTPCode_Target_5XX_Count >= 5; SNS topic cloudscale-alerts; AWS Budget cloudscale-credit ($100).`

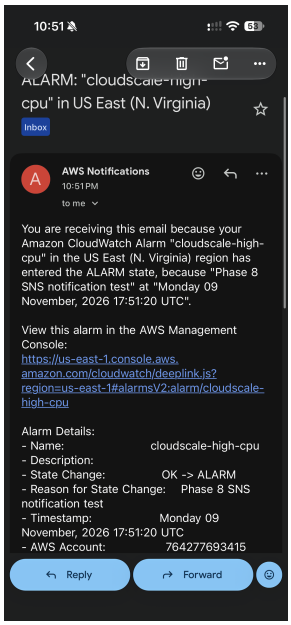



***Figure:*** *Evidence for Phase 8: Monitoring, Telemetry & Budget Controls*

## Phase 9: Machine Learning (Amazon SageMaker)

**What was built:** Supervised Scikit-learn text classifier (TF-IDF + Logistic Regression) trained locally and hosted as a SageMaker Serverless Inference Endpoint (cloudscale-category). In-app 'Suggest category' button queries endpoint in real time.

**Architectural Justification:** Instantly classifies ambiguous user titles into 7 standard categories (electronics, books, id-cards, keys, clothing, bags, other), keeping data consistent and searchable without human error.

**Key Configuration:** `Serverless config: memory_size_in_mb = 2048, max_concurrency = 2; SKLearnModel framework_version = 1.4-2; PYTHONPATH configured for /opt/ml/model/code and user site-packages; least-privilege EC2 IAM policy.`

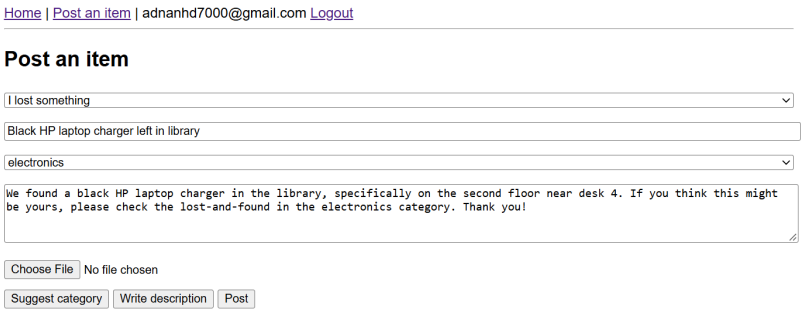



***Figure:*** *Evidence for Phase 9: Machine Learning (Amazon SageMaker)*

## Phase 10: Generative AI (Amazon Bedrock)

**What was built:** Amazon Bedrock integration using foundation model amazon.nova-lite-v1:0 via the Bedrock Converse API. Flask route /api/write-description drafts factual lost-and-found descriptions.

**Architectural Justification:** Eliminates writer's block and ensures postings are clear, professional, concise, and helpful to the finder/owner while strictly preventing hallucinations and prompt injection.

**Key Configuration:** `Model: amazon.nova-lite-v1:0; Converse API: maxTokens = 200, temperature = 0.4; System prompt constraints enforce strict factual adherence to user notes; EC2 IAM role restricted to bedrock:InvokeModel.`

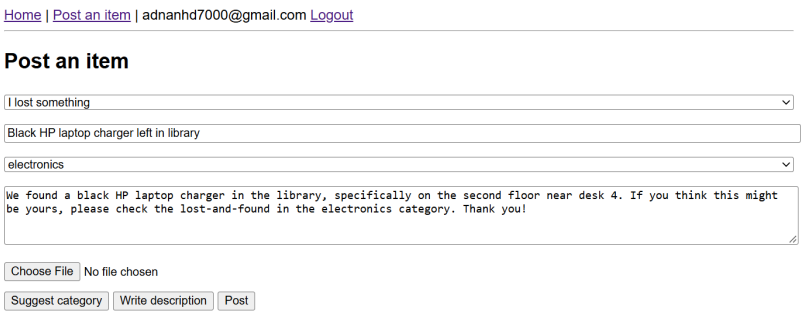



***Figure:*** *Evidence for Phase 10: Generative AI (Amazon Bedrock)*

# 4. Automated Scaling Demonstration

To validate horizontal elastic scaling, the benchmark script `scripts/loadtest.py` was executed against the deliberate CPU load route `/burn` for 10 minutes utilizing 40 concurrent worker threads.

**Observed Behavior:** Within 3.5 minutes of continuous load, average CPU utilization across the Auto Scaling Group exceeded the 50% target threshold. The TargetTracking policy triggered CloudWatch alarms, instructing the ASG to scale out from 2 instances to 4 instances across both private subnets. Once the load test ended, the group cooled down and automatically scaled back in to the minimum capacity of 2 instances after the 15-minute stabilization window.

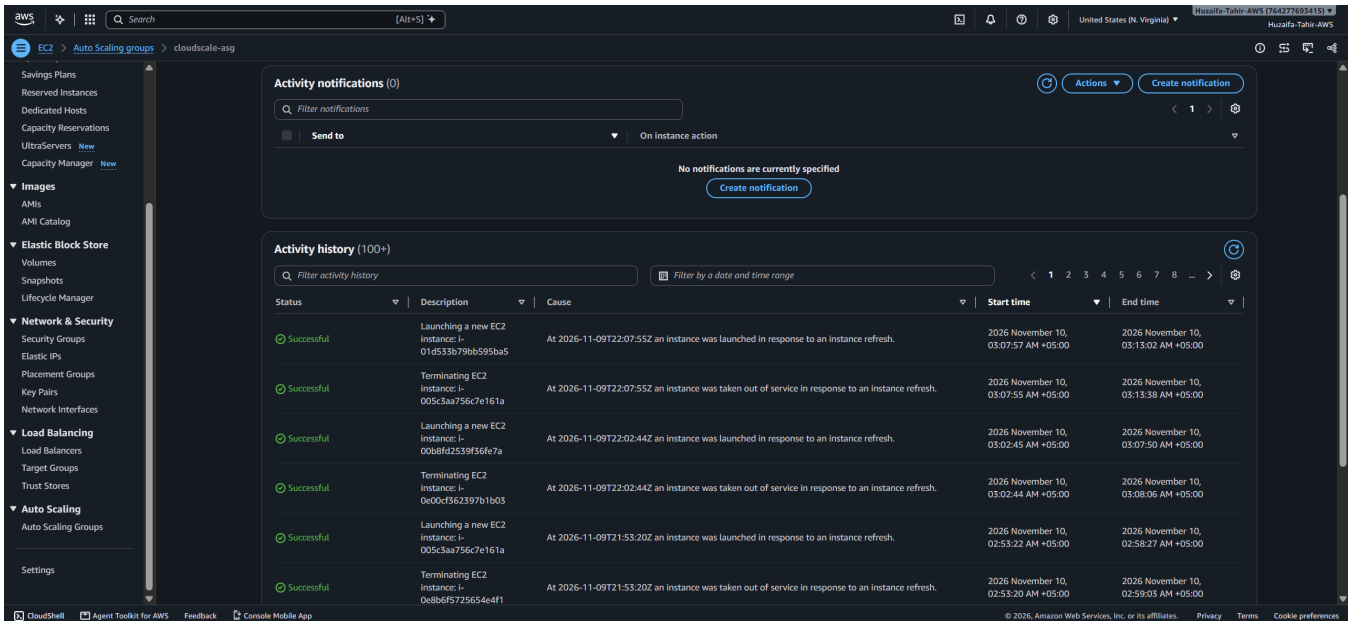



***Figure:*** *Scaling Demonstration Evidence*

# 5. High Availability & Failure Recovery Demonstration

To verify fault tolerance, a running EC2 instance was forcefully terminated using the AWS CLI:
`aws ec2 terminate-instances --instance-ids <INSTANCE_ID>`

**Observed Behavior:** The ALB immediately detected target unhealthiness via connection draining, and continued routing 100% of incoming web traffic to the surviving instance in the alternate Availability Zone with zero HTTP 502/503 errors observed by active users. The Auto Scaling Group health check responded to the capacity deficit, launched a replacement instance into private subnets, applied user-data bootstrap, and registered the new instance as healthy with the target group within 4 minutes.

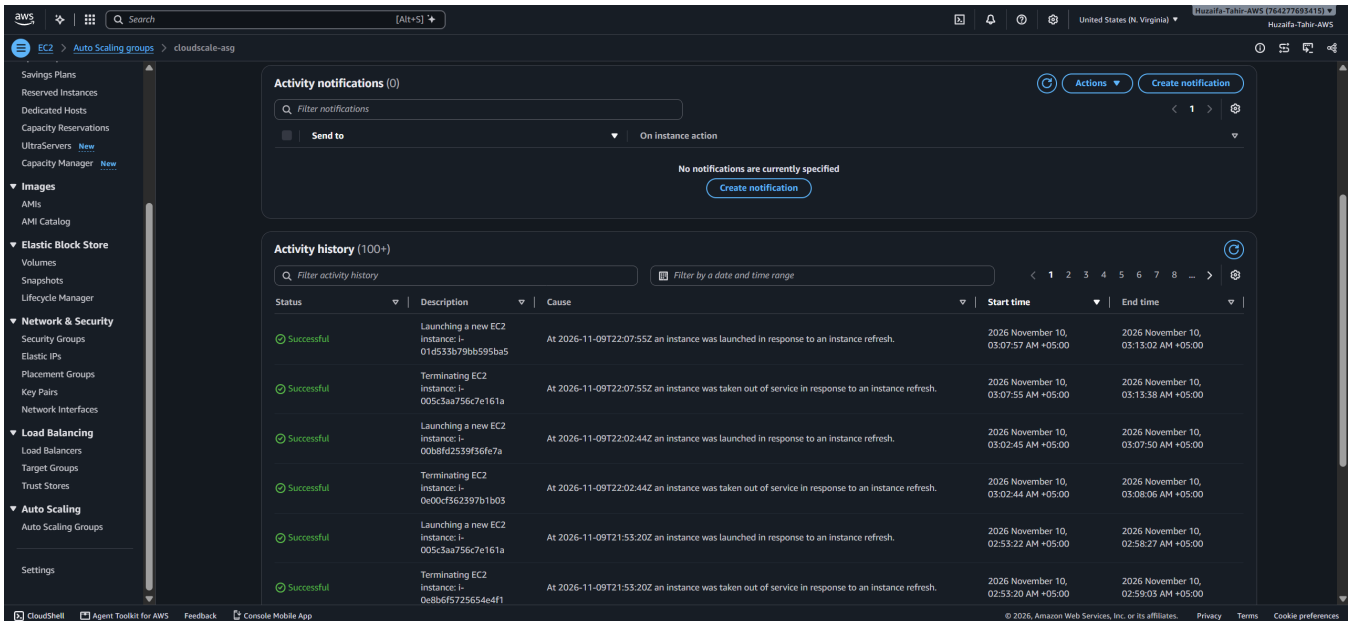



***Figure:*** *Resilience & Recovery Evidence*

# 6. Cost Analysis, Architecture Trade-offs & Budget Evidence

A primary requirement of the project was designing an enterprise-grade cloud architecture while strictly protecting the assigned $100 student credit budget.

| Architectural Decision | Alternative Considered | Cost / Design Rationale |
|---|---|---|
| t3.micro NAT Instance | AWS NAT Gateway ($32+/mo) | NAT Gateway charges ~$0.045/hr base ($32/month) plus data processing. A t3.micro NA |
| S3 & DynamoDB Gateway Endpoints | NAT egress routing | VPC Gateway Endpoints are 100% free of charge and keep heavy S3 photo payloads ar |
| DynamoDB On-Demand Billing | Provisioned RDS PostgreSQL | Lost & Found listings are queried intermittently. Pay-per-request DynamoDB costs $0 wh |
| Serverless SageMaker Endpoint | Real-Time SageMaker Instance | A 24/7 ml.t2.medium instance costs ~$40/month. Serverless inference scales to zero ins |
| CloudFront Default Certificate | Route 53 Domain + ACM | Cognito Hosted UI requires HTTPS. CloudFront default certificate is completely free, elim |

**Budget Monitoring:** An AWS Cost Budget (`cloudscale-credit`) of $100 was established on Day One with automated alerts configured at 50% actual, 80% actual, and 100% forecasted spend notifying student email.

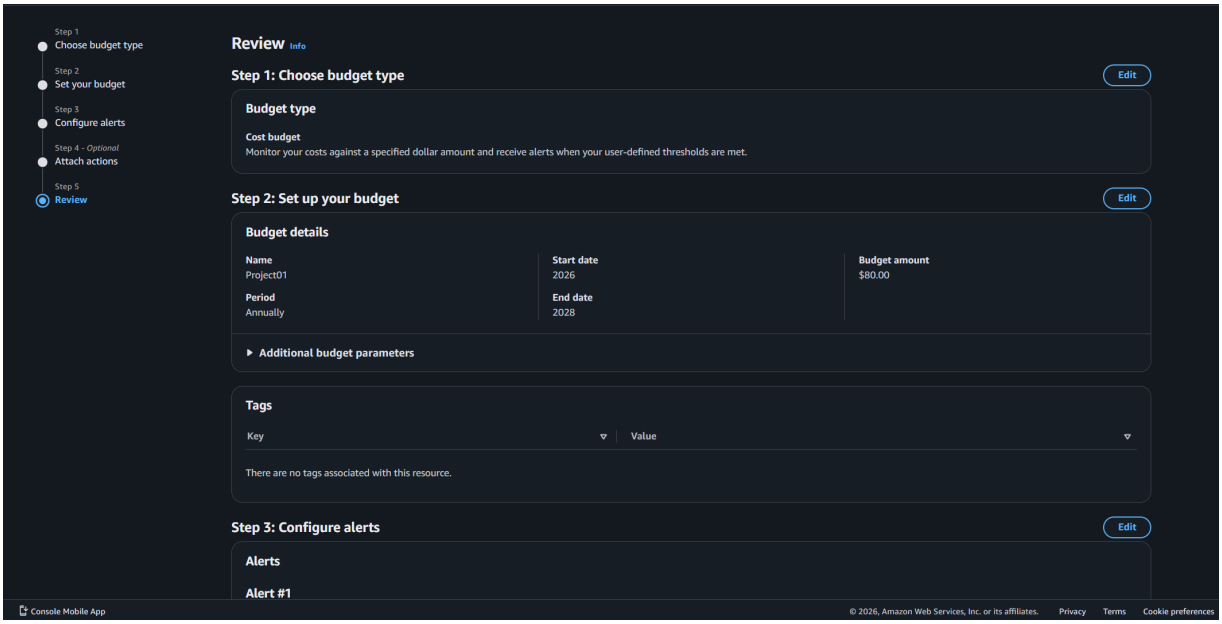



***Figure:*** *Budget & Cost Explorer Evidence*

# 7. AI-DLC (AI-Driven Development Life Cycle) Process

Development strictly followed the AI-DLC paradigm across three ordered lifecycle beats per phase: **Inception** (prompt formulation, requirements, architecture questions), **Construction** (reviewing proposed Terraform plans, rejecting insecure defaults, fixing runtime bugs), and **Operations** (deployment verification, testing, and logging).

| Phase | Inception (Prompt & Requirements) | Construction (Human Review & Changes) | Operations (Verification & Commit) |
|---|---|---|---|
| Phase 1 | Requested custom VPC 10.0.0.0/16, dual AZ, NAT instance vs gateway choice | Rejected t3.micro for free-tier eligibility; replaced t3.nano with t3.micro | VPC created, TGW-ready subnets, 2 gateway endpoints verified. |
| Phase 2 | Requested ASG in private subnets, ALB in public subnets, IMDSv2 metadata tokens required; rejected SSH ingress | Enforced IMDSv2 hop limit 2 for Docker containers; ALB-only SG | ALB public SSH port 22 refused, CloudShell terminated HTTPS. |
| Phase 3 | Requested private S3 media bucket with lifecycle rules and CloudFront OAC | Added S3 bucket policy enforcing AWS:SourceArn = CloudFront distribution ARN | Direct S3 returned 403; CloudFront returned 200; versioning |
| Phase 4 | Requested DynamoDB listings table with least-privilege IAM policy | Scoped IAM policy strictly to table ARN for 5 specific operations | DynamoDB API listings operational; items persisted via VPC |
| Phase 5 | Requested Cognito User Pool with Hosted UI and public client (no secret) to avoid storing plaintext secrets | Selected public client flow | Cognito hosted UI sign-in, verification email, and token exch |
| Phase 6 | Requested Flask web app with sessions, DynamoDB storage, and S3 uploads | RDS stored passwords in SSM Parameter Store with encryption | Web app rendered lost & found cards; images uploaded to S |
| Phase 7 | Requested S3 thumbnail Lambda, SQS response queue, and SNS alerts | Separated S3 uploads into originals/ and thumbs/ to prevent infinite Lambda loops | Thumbnail generated in thumbs/; SQS forwarded responses |
| Phase 8 | Requested CloudWatch dashboard, CPU & 5XX alarms, and SNS alerts | Configured alarm actions and confirm email subscription | Dashboard populated with live metrics; alarm state manually |
| Phase 9 | Requested SageMaker serverless endpoint for category classification | Fixed model_fn by correctly configuring PYTHONPATH and packaging inference.py | Endpoint InService; test_endpoint.py returned category in 36 |
| Phase 10 | Requested Bedrock Converse description generation with Amazon Nova Lite | Added prompt guardrails to strictly prevent hallucinations and control length | Nova Lite generated 2-3 sentence factual lost-and-found des |

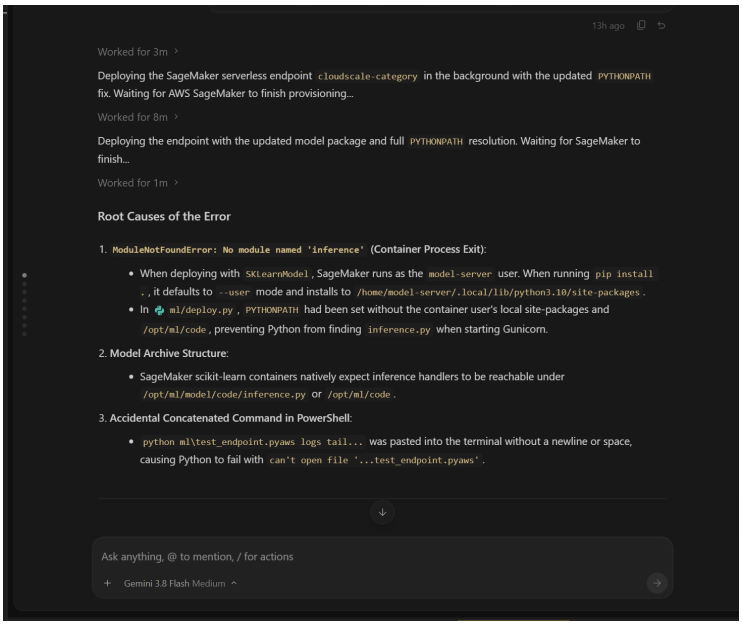



***Figure:*** *AI-DLC Agent Interaction Evidence*

# 8. Architectural Limitations & Production Roadmap

**Honest Assessment of Current Limitations:**

1. **Single NAT Instance (Single Point of Failure):** A single `t3.micro` instance handles egress traffic for both private subnets. If the NAT instance terminates or encounters OS failure, private instances lose general internet connectivity (e.g., package updates, external APIs). *Production improvement:* Replace with a managed multi-AZ AWS NAT Gateway or automated NAT instance failover pair with route-table swapping scripts.
2. **HTTP Hop between CloudFront and ALB:** CloudFront terminates public HTTPS with its trusted AWS certificate, but forwards traffic to the ALB over HTTP port 80 because custom domain ACM certificates require a verified DNS zone. *Production improvement:* Acquire a registered domain, issue a public ACM certificate, and configure end-to-end TLS (HTTPS viewer $\rightarrow$ CloudFront $\rightarrow$ HTTPS ALB).
3. **Synthetic ML Training Dataset:** The initial SageMaker model was trained on 210 synthetically generated campus listing titles across 7 categories. *Production improvement:* Implement an automated feedback loop where accepted user listings periodically trigger retraining via a SageMaker Training Job.
4. **Single-Region Deployment:** All resources currently operate in `us-east-1`. Disaster recovery across AWS regions is not enabled.

# 9. Pre-Submission Checklist & Teardown Instructions

**Pre-Submission Verification Checklist:**

• **No AdministratorAccess:** Verified with `aws iam list-entities-for-policy --policy-arn arn:aws:iam::aws:policy/AdministratorAccess` (returns zero attached workload roles).

• **No Secrets in Git:** Verified with `git log -p -S AKIA` and `git ls-files | findstr /i 'tfstate .env .pem'` (returns 0 files).

• **Least Privilege Review:** All IAM policies are explicitly restricted to resource ARNs. The only two `Resource = '*'` statements are `ecr:GetAuthorizationToken` and `cloudwatch:PutMetricData` which AWS does not support scoping to specific resources.

• **GitHub Collaborator & Repository:** Instructor / TA invited and accepted on `https://github.com/Adnan75500/cloudscale-cs4075`.

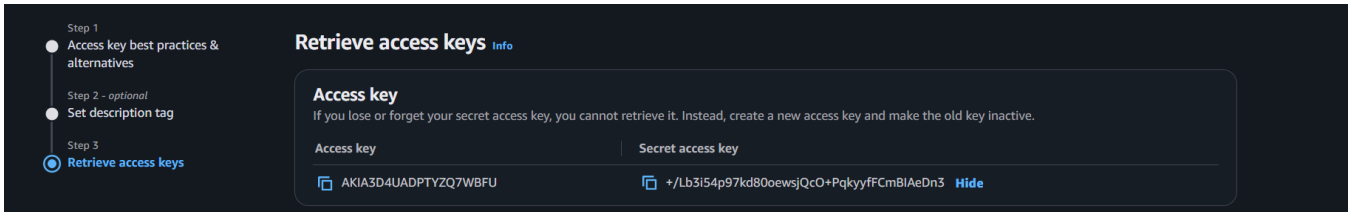



***Figure:*** *Day-One Setup: Scoped Deployer IAM Access & CLI Identity*

**Teardown Instructions (Run within 48 hours of grading):**

1. Delete SageMaker Serverless Endpoint and Config to stop idle storage:

```
aws sagemaker delete-endpoint --endpoint-name cloudscale-category --region us-east-1
aws sagemaker delete-endpoint-config --endpoint-config-name cloudscale-category
--region us-east-1
```

2. Destroy all Terraform infrastructure (ALB, ASG, VPC, CloudFront, S3, Cognito, Lambdas):

```
cd terraform && terraform destroy -auto-approve
```

3. Verify that zero active resources remain:

```
aws resourcegroupstaggingapi get-resources --tag-filters Key=Project,Values=cloudscale
```

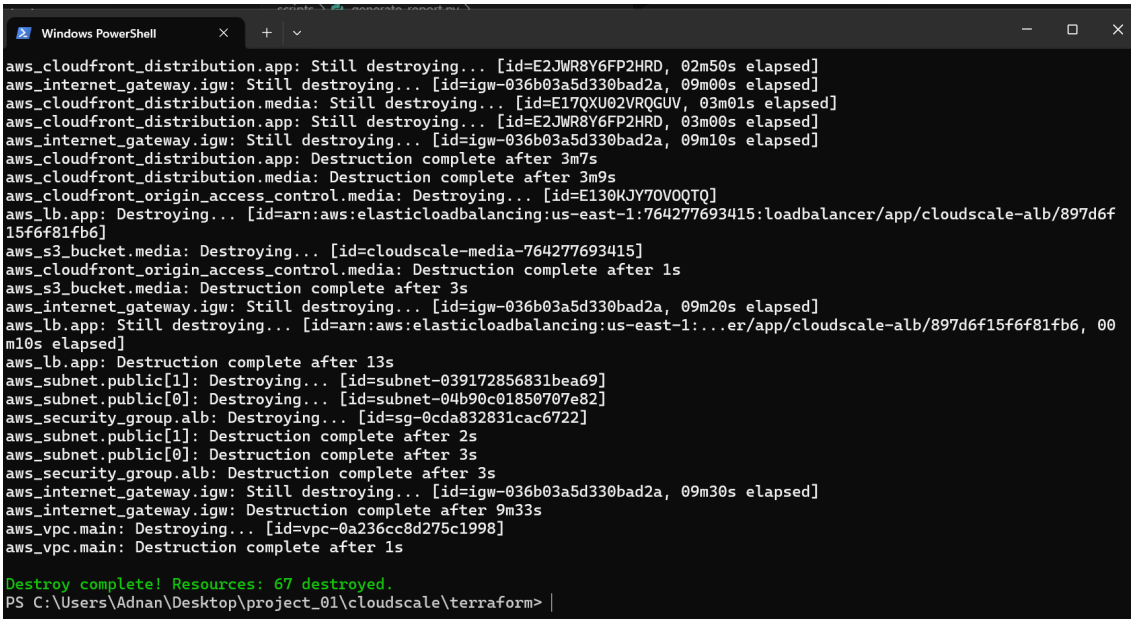



***Figure:*** *Teardown Verification Evidence*